Mathematical Formulas for Quad Persona System with Dynamic Knowledge Mapping

Mathematical Formulas for Quad Persona System

1. Core System Architecture

1.1 Quad Persona Framework Definition

The Quad Persona System is defined as a composite function operating over a multi-dimensional knowledge space, represented as:

$$QPS(q, c, t) = \Psi(KE(q, c, t), SE(q, c, t), RE(q, c, t), CE(q, c, t))$$

Where:

- q: input query vector
- c: context vector
- t: temporal parameter
- KE: Knowledge Expert function
- SE: Sector Expert function
- RE: Regulatory Expert function
- CE: Compliance Expert function
- Ψ: integration function that synthesizes the outputs of all four personas

1.2 Multi-dimensional Knowledge Space

The knowledge space Ω is defined as a 13-dimensional manifold:

$$\Omega = \{(p, l, b, h, s, o, loc, t, r, c, e, m, k) \in \mathbb{R}^{13}\}$$

Where each dimension represents:

- p: pillar dimension (1-87 pillar levels)
- l: level within pillar
- b: branch dimension
- h: honeycomb dimension (cross-connections)
- s: spiderweb dimension (compliance connections)
- o: octopus dimension (regulatory connections)
- loc: location dimension
- t: temporal dimension
- r: risk dimension
- c: confidence dimension
- e: ethical dimension
- m: methodology dimension
- k: knowledge level dimension

2. Persona Functions

2.1 Knowledge Expert Function

The Knowledge Expert function processes queries from a theoretical knowledge perspective:

$$KE(q, c, t) = \sum_{i=1}^{n} \alpha_i(t) \cdot f_{KE}(q, c, \omega_i)$$

Where:

- α_i(t): time-dependent weight for knowledge component i
- f_KE: knowledge processing function
- ω_i: knowledge node at position i in the knowledge space

2.2 Sector Expert Function

The Sector Expert function processes queries from an industry-specific perspective:

$$SE(q, c, t) = \sum_{j=1}^{m} \beta_j(c) \cdot f_{SE}(q, c, \sigma_j)$$

Where:

- β_j(c): context-dependent weight for sector component j
- f_SE: sector processing function
- σ_j: sector node at position j in the knowledge space

2.3 Regulatory Expert Function

The Regulatory Expert function processes queries from a legal and regulatory perspective:

$$RE(q, c, t) = \sum_{k=1}^{p} \gamma_k(c, t) \cdot f_{RE}(q, c, \rho_k)$$

Where:

- γ_k(c, t): context and time-dependent weight for regulatory component k
- f_RE: regulatory processing function
- ρ_k: regulatory node at position k in the knowledge space

2.4 Compliance Expert Function

The Compliance Expert function processes queries from a standards and compliance perspective:

$$CE(q, c, t) = \sum_{l=1}^{q} \delta_l(c, t) \cdot f_{CE}(q, c, \kappa_l)$$

Where:

- δ_l(c, t): context and time-dependent weight for compliance component l
- f_CE: compliance processing function

- κ_l: compliance node at position l in the knowledge space

2.5 Integration Function

The integration function synthesizes outputs from all four personas:

$$\Psi(KE, SE, RE, CE) = w_{KE} \cdot KE + w_{SE} \cdot SE + w_{RE} \cdot RE + w_{CE} \cdot CE$$

Where w_KE, w_SE, w_RE, and w_CE are dynamic weights determined by context relevance, confidence scores, and criticality measures.

3. Dynamic Knowledge Mapping

3.1 Knowledge Mapping Function

The dynamic knowledge mapping function maps queries to relevant points in the knowledge space:

$$M(q, c, t) = \{(p_i, s_i) | p_i \in \Omega, s_i = sim(q, p_i, c, t) > \theta\}$$

Where:

- p_i: point in the knowledge space
- s_i: similarity score between query and point
- sim(): similarity function
- θ: relevance threshold

3.2 Pillar Selection Function

For a given query, the pillar selection function identifies the most relevant knowledge pillars:

$$PS(q, c) = \arg\max_{p \in P} \sum_{i=1}^{d} w_i \cdot sim_i(q, p_i, c)$$

Where:

- P: set of all pillars
- w_i: weight for dimension i

- sim_i(): similarity function for dimension i

3.3 Cross-dimensional Mapping

The cross-dimensional mapping function connects knowledge across different dimensions:

$$CDM(p_1, p_2) = \frac{\sum_{i=1}^{k} \omega_i \cdot sim_i(p_1, p_2)}{\sqrt{\sum_{i=1}^{k} \omega_i^2}}$$

Where:

- p_1, p_2: points in the knowledge space
- ω_i: weight for similarity measure i
- sim_i(): similarity function for measure i

3.4 Dynamic Weight Adaptation

Weights in the system adapt based on performance feedback:

$$w_i(t+1) = w_i(t) + \eta \cdot \nabla_w L(w_i(t))$$

Where:

- η: learning rate
- ∇_w L(): gradient of the loss function with respect to weight

4. Structured Memory Recall Algorithms

4.1 Memory Structure

The memory structure is defined as a graph:

$$G_M = (V_M, E_M, A_M)$$

Where:

- V_M: set of memory vertices
- E_M: set of memory edges

- A_M: set of attributes

4.2 Memory Access Function

The memory access function retrieves relevant memories:

$$MA(q, c, t) = \{m_i \in M | sim(q, m_i, c, t) > \theta_M\}$$

Where:

- M: set of all memories
- sim(): similarity function
- θ_M: memory retrieval threshold

4.3 Structured Recall Algorithm

The structured recall algorithm combines temporal and relevance factors:

$$SR(q, c, t) = \sum_{i=1}^{|M|} [R(m_i, q, c) \cdot T(m_i, t) \cdot I(m_i)] \cdot m_i$$

Where:

- R(): relevance function
- T(): temporal importance function
- I(): importance function

4.4 Memory Consolidation Function

The memory consolidation function integrates new information:

$$MC(M, I, t) = M \cup \{m_{new} | m_{new} = f_{MC}(I, M, t)\}$$

Where:

- I: new information
- f_MC: memory creation function

5. Deep Recursive Learning

5.1 Recursive Processing Function

The recursive processing function applies iterative refinement:

$$RPF(x, d) = \begin{cases} x & \text{if } d = 0 \\ g(RPF(x, d-1)) & \text{if } d > 0 \end{cases}$$

Where:

- x: input data
- d: recursion depth
- g(): refinement function

5.2 Recursive Confidence Evaluation

The recursive confidence evaluation function measures certainty:

$$RCE(x, d) = 1 - \frac{1}{1 + e^{\lambda \cdot (d - \theta_d)}}$$

Where:

- λ: steepness parameter
- θ_d: threshold parameter

5.3 Deep Recursive Learning Algorithm

The deep recursive learning algorithm combines multiple recursive steps:

$$DRL(x, c, D) = RPF_D(RPF_{D-1}(...RPF_1(x, c)...))$$

Where:

- RPF_i: recursive processing function at step i
- D: maximum recursion depth

5.4 Convergence Function

The convergence function determines when recursion should stop:

$$CF(x_t, x_{t-1}, \epsilon) = \begin{cases} 1 & \text{if } ||x_t - x_{t-1}|| < \epsilon \\ 0 & \text{otherwise} \end{cases}$$

Where:

- x_t: result at iteration t
- ε: convergence threshold

6. 12-Step Refinement Workflow

6.1 Mathematical Representation

The 12-step refinement workflow is defined as a composition of functions:

$$RW_{12} = f_{12} \circ f_{11} \circ ... \circ f_1$$

Where each function f_i represents a specific refinement step:

- f_1: Tree of Thought (ToT)
- f_2: Algorithm of Thought (AoT)
- f_3: Gap Analysis
- f_4: Knowledge Verification
- f_5: NLP Enhancement
- f_6: Data Consistency Validation
- f_7: Ethical Analysis
- f_8: Bias Audit
- f_9: Security Check
- f_10: Logic Verification
- f_11: Compliance Check
- f_12: Final Optimization

6.2 Confidence Threshold Function

The confidence threshold function determines if the result meets the confidence requirement:

$$CT(x) = \begin{cases} 1 & \text{if } conf(x) \geq 0.995 \\ 0 & \text{otherwise} \end{cases}$$

Where conf(x) is the confidence score function.

7. Integrated System Formulation

7.1 Complete System Function

The complete Quad Persona System with dynamic knowledge mapping and structured memory recall is defined as:

$$QPS_{full}(q, c, t) = RW_{12}(DRL(QPS(q, c, t), c, D_{max}))$$

Where:

- QPS(): Quad Persona System function
- DRL(): Deep Recursive Learning function
- RW_12(): 12-step refinement workflow
- D_max: maximum recursion depth

7.2 System Optimization Function

The system optimization function improves performance over time:

$$OPT(QPS, H) = \arg\min_{\Theta} \sum_{(q_i, a_i) \in H} L(QPS(q_i, c_i, t_i; \Theta), a_i)$$

Where:

- H: historical query-answer pairs
- Θ: system parameters
- L(): loss function

7.3 Dynamic Knowledge Adaptation Function

The dynamic knowledge adaptation function updates the knowledge space:

$$\Omega_{t+1} = \Omega_t + \eta \cdot \nabla_{\Omega} L(\Omega_t)$$

Where:

- Ω_t: knowledge space at time t
- η: learning rate
- ∇_Ω L(): gradient of the loss function with respect to the knowledge space

8. Algorithm Implementation

```
def quad_persona_system(query, context, time):
    """
    Implements the Quad Persona System with dynamic knowledge mapping an
d
    structured memory recall using deep recursive learning.

    Args:
        query: The input query vector
        context: The context vector
        time: The temporal parameter

    Returns:
        The processed response
    """
    # 1. Map query to knowledge space
    relevant_points = knowledge_mapping(query, context, time)

    # 2. Apply each persona function
    ke_output = knowledge_expert(query, context, time, relevant_points)
    se_output = sector_expert(query, context, time, relevant_points)
    re_output = regulatory_expert(query, context, time, relevant_points)
    ce_output = compliance_expert(query, context, time, relevant_points)
```

```
    # 3. Integrate persona outputs
    initial_output = integrate_personas(ke_output, se_output, re_output, ce_outp
ut)

    # 4. Apply deep recursive learning
    refined_output = deep_recursive_learning(initial_output, context, max_depth
=12)

    # 5. Apply 12-step refinement workflow
    final_output = apply_refinement_workflow(refined_output)

    # 6. Check confidence threshold
    if confidence_score(final_output) >= 0.995:
        return final_output
    else:
        return deep_recursive_learning(final_output, context, max_depth=12)
```

9. Conclusion

The mathematical formulas presented in this document provide a comprehensive framework for implementing the Quad Persona System with dynamic knowledge mapping and structured memory recall via deep recursive learning. The system integrates four specialized expert personas, each contributing unique perspectives to query processing. The dynamic knowledge mapping enables efficient navigation of the multi-dimensional knowledge space, while the structured memory recall algorithms ensure relevant information retrieval. The deep recursive learning mechanism, combined with the 12-step refinement workflow, drives the system toward high-confidence responses.

This mathematical framework serves as the foundation for implementing a sophisticated AI system that can effectively simulate multiple expert perspectives, navigate complex knowledge structures, and continuously improve through recursive self-application.